

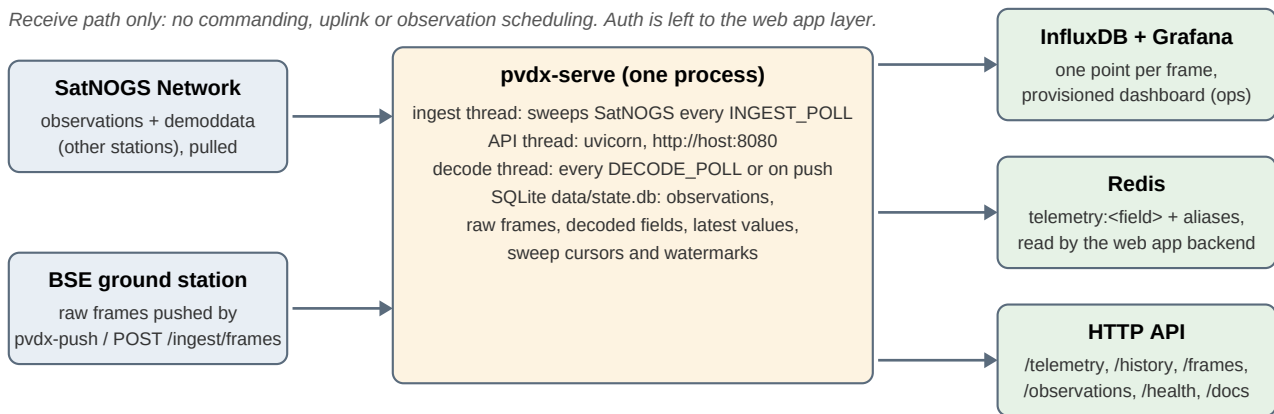
PVDX Ground Pipeline

Operations Guide: SatNOGS + ground-station ingest, decode, storage, HTTP API, Redis and dashboard

Brown Space Engineering · version 0.2.0 · 2026-09-27

This guide explains how to run and look after the cloud telemetry service: it pulls demodulated frames for one satellite from the SatNOGS Network, accepts frames pushed by our own ground station, stores them with provenance, decodes them, and serves the result to the mission-control web app (HTTP API and Redis) and to operators (Grafana). It assumes the repository at `/Users/bryan/code/bse_code` (adjust paths for another checkout). Until PVDX launches the service runs against a stand-in satellite; section 9 covers the swap.

Receive path only: no commanding, uplink or observation scheduling. Auth is left to the web app layer.



1. What runs where

A working installation is two things:

- **Three containers** started by Docker Compose: InfluxDB 2.7 on port 8086 (time series), Grafana 11.3 on port 3000 (dashboard, provisioned automatically with the datasource and the dashboard) and Redis 7 on port 6379 (the latest-values cache the web app backend reads).
- **The service** (`pvd-serve`): one Python process with three parts. The **ingest** thread sweeps the SatNOGS observations list for the configured NORAD id and downloads every demodulated frame file into `data/state.db` (SQLite). The **decode** thread decodes new frames, writes one InfluxDB point per frame, keeps the decoded fields in SQLite and publishes the latest values to Redis. The **HTTP API** on port 8080 serves telemetry, history, frames, observations and health, and accepts frames pushed by our ground station (which are decoded within a second).

The service runs on your machine or, with the `cloud compose` profile, as a fourth container (section 3).

The one-shot commands `pvd-ingest` and `pvd-decode` still exist for operations and are safe to re-run at any time: nothing is downloaded twice and nothing is skipped. Never run `pvd-serve` and a polling `pvd-ingest` against the same `data/state.db` at once. If no service is running, the dashboard is static and the Redis keys expire.

2. One-time setup

2.1 Requirements

- macOS or Linux with **Python 3.11 or newer** (uv recommended, plain venv/pip works).
- **Docker Desktop** with Compose v2 (docker compose version must work). Docker Desktop has to be running before docker compose up.
- A **SatNOGS Network API key**, optional but strongly recommended (see 2.3).

2.2 Install

```
cd /Users/bryan/code/bse_code
cp .env.example .env          # first time only; then edit .env (see 2.3 and 2.4):
                                # token, INFLUX_* secrets, INGEST_START, INGEST_TOKEN

uv venv --python 3.12 .venv
source .venv/bin/activate
uv pip install -e ".[dev]"    # or: pip install -e ".[dev]"
pytest                       # 80 tests, no network access needed
```

Every command in this guide assumes the virtual environment is active (source .venv/bin/activate) and the working directory is the repository root, because .env and data/ are resolved relative to it.

2.3 The SatNOGS token

SatNOGS **Network** and SatNOGS **DB** issue separate tokens. The pipeline needs the Network one, shown under "API Key" at <https://network.satnogs.org/users/edit/>. A DB token is rejected by the Network API with 401 Invalid token; pvdX-ingest logs that once and continues anonymously, because reads work without a token. The difference is speed: the observations list is throttled to **60 requests per hour anonymously and 240 per hour with a valid token**, and each request returns 25 observations. The token is only ever sent to the SatNOGS host, never to the object storage that serves frame files.

2.4 Configuration reference (.env)

Variable	Default	Meaning
SATNOGS_API_TOKEN	(empty)	SatNOGS Network API key. Empty means anonymous access.
SATNOGS_NETWORK_URL	https://network.satnogs.org	API base URL.
NORAD_CAT_ID	(required)	Satellite to ingest. SatNOGS temporary ids (98xxx/99xxx) also work.
INGEST_START	now minus 7 days if unset (.env.example pins a fixed date)	Floor for every sweep: a sweep starts at max(INGEST_START, watermark minus overlap), so in practice only the first sweep is affected. Keep it a fixed date a few days back; an old date triggers a multi-week first backfill.
INGEST_OVERLAP_HOURS	48	How far behind the watermark each later sweep re-scans.
INGEST_STATUS	good	Observation status filter; frames only exist on good observations.
INGEST_MIN_INTERVAL	from throttle	Seconds between list requests (63 anonymous, 15.8 with token).

Variable	Default	Meaning
STATE_DB	data/state.db	SQLite file with observations, raw frames and sweep state.
DECODER	(empty)	satnogs:<struct> (e.g. satnogs:crocube) or pvdX.
INFLUX_URL / ORG / BUCKET	http://localhost:8086 / bse / telemetry	InfluxDB connection used by pvdX-decode. ORG and BUCKET (not URL) are also read by docker compose to initialise the container. Keep the bucket named telemetry: the dashboard JSON hardcodes bucket and measurement names.
INFLUX_TOKEN	(set one)	Admin token you choose; the InfluxDB container registers it on its first start and pvdX-decode and Grafana authenticate with it.
INFLUX_USERNAME / PASSWORD	admin / (set one, 8+ characters)	InfluxDB UI login at localhost:8086. InfluxDB refuses shorter passwords on first start; the container then exits and Grafana never starts.
INFLUX_RETENTION	0 (forever)	Retention of the telemetry bucket, applied by docker compose when the InfluxDB volume is first initialised (e.g. 30d). Not read by the Python commands.
GRAFANA_ADMIN_PASSWORD	admin	Grafana login (user admin).
API_HOST / API_PORT	127.0.0.1 / 8080	Where pvdX-serve listens. 0.0.0.0 inside a container.
API_CORS_ORIGINS	http://localhost:3000	Browser origins allowed to call the API directly (comma separated, or *).
INGEST_TOKEN	(empty)	Shared secret the ground station must send as X-Ingest-Token on POST /ingest/frames. Empty accepts pushes from anyone (a warning is logged).
INGEST_POLL / DECODE_POLL	600 / 60	Seconds between SatNOGS sweeps and between decode passes in pvdX-serve. Pushed frames are decoded at once.
TELEMETRY_STALE_AFTER	3600	GET /telemetry reports stale=true when the newest decoded frame is older than this many seconds.
TELEMETRY_ALIASES	(empty)	Extra names for decoded fields, alias=field,alias=field. .env.example maps the web app's battery/temperature/signal_rssi/uptime_seconds to CroCube fields.
REDIS_URL	(empty = off)	redis://localhost:6379/0 for the compose Redis. Latest values are published after every decode pass.
REDIS_KEY_PREFIX / REDIS_TELEMETRY_TTL	telemetry: / 3600	Key prefix (the web app reads telemetry:<name>) and lifetime of the keys; expired keys make the web app show stale.
PUSH_URL / PUSH_STATION / PUSH_TOKEN	(empty)	Only for pvdX-push on the ground station computer: service URL, this station's name, and the service's INGEST_TOKEN.

Any pvdX-serve, pvdX-ingest, pvdX-decode or pvdX-push flag overrides the file for that run (section 5). The InfluxDB credentials are baked into the containers' data volume on first start; if you change them later, wipe the volumes (section 8.5) or the containers will keep the old ones.

3. Starting and stopping the stack

```

docker compose up -d          # start InfluxDB + Grafana + Redis (pulls images the first time)
docker compose ps             # all three Up; influxdb and redis show (healthy)
open http://localhost:3000    # Grafana login: admin / GRAFANA_ADMIN_PASSWORD; click "Skip"
                              # on the "Update your password" prompt (or set a new one)

docker compose down           # stop; data volumes are kept
docker compose down -v        # stop AND delete all InfluxDB/Grafana data (Redis keeps nothing anyway)

docker compose --profile cloud up -d --build # ALSO build + run the service as the "pvdX" container
docker compose --profile cloud logs -f pvdX  # its log; state stays in ./data, API on :8080
docker compose --profile cloud down          # stop everything including the service

```

The pvdX container reads `.env`, replaces the InfluxDB and Redis URLs with the compose service names, mounts `./data` for the state database and restarts unless stopped. Use it or a host-run `pvdX-serve`, never both on the same `data/state.db`.

The provisioned dashboard is at `http://localhost:3000/d/pvdX-telemetry` in the **PVDX** folder. Grafana re-reads `grafana/dashboards/*.json` every 30 seconds, so edits to the JSON file appear without a restart. InfluxDB's own UI at `http://localhost:8086` is available for ad-hoc Flux queries.

4. Running the service

4.1 The service: pvdX-serve

```

pvdX-serve          # ingest every INGEST_POLL s, decode every DECODE_POLL s (or on push),
                    # publish to Redis, API on http://127.0.0.1:8080 (docs at /docs). Ctrl-C s
pvdX-serve --no-ingest # frames arrive by push only (no SatNOGS sweeps)
pvdX-serve --port 9000 -v # other port; debug log + HTTP access log
curl -s localhost:8080/health # status ok/degraded, worker threads, InfluxDB and Redis reachability

```

Start the Docker stack first. The decode part verifies the InfluxDB token, org and bucket at startup and keeps retrying if they are wrong or unreachable; Redis being down is logged and retried on the next pass. The first start after an upgrade from 0.1 migrates `data/state.db` to schema 2 and re-decodes the frames that were already decoded, so their decoded fields get stored (the InfluxDB points are rewritten identically). Under `launchd/systemd` give the service a restart-on-failure policy; `GET /health` reports a dead worker thread as degraded.

4.2 One-shot commands

```

pvdX-ingest          # one sweep: new observations + frames -> data/state.db
pvdX-decode           # decode new frames, write points to InfluxDB (+ Redis if REDIS_URL is set)
pvdX-ingest --stats   # counters: observations, frames ok/pending/failed/decoded, watermark
pvdX-ingest --poll 600 # the two loops as separate processes instead of pvdX-serve
pvdX-decode --poll 60

```

These are the same code paths the service runs; use them for a single sweep, a re-decode, or on a machine without the API. Stop them with Ctrl-C, which is safe at any moment.

4.3 What to expect on the first run

- The first sweep covers `INGEST_START` to now. Each API page holds 25 observations and the client waits about 63 s between pages anonymously (16 s with a token), so a week of an active satellite (several hundred observations) takes tens of minutes of paging.
- Frames are downloaded after every page, so data flows early. The stand-in satellite delivers up to ~100 frames per pass; at ~0.25 s per file a busy page can take 10 minutes to download.

- Use `--max-pages N` to ration a run. A run stopped by `--max-pages`, Ctrl-C or a crash leaves a resumable sweep; the next run continues from the saved cursor and does not re-fetch stored pages.
- When a sweep reaches its last page it is marked complete and the satellite's watermark advances to the time that sweep was opened. Later sweeps start at watermark minus `INGEST_OVERLAP_HOURS` (but never earlier than `INGEST_START`), which picks up observations vetted late and frames uploaded late.
- Check `INGEST_START` before the first run: at one page per minute plus frame downloads, a multi-week backfill of an active satellite takes hours. A few days is plenty for a first look.

4.4 Reading the log lines

```
starting sweep 3 for NORAD 62394: status=good since 2026-09-24T12:00:00Z (anonymous)
page 1: 25 observation(s), 4 new, 380 new frame(s) queued
--max-pages 1 reached; sweep 3 will resume next run
sweep 3 for NORAD 62394 paused (will resume): 1 page(s), 25 observations (4 new), ...
decode: 2433 frame(s) read, 91 decoded, 91 written, 0 rejected by InfluxDB, 0 empty,
      2342 not telemetry/undecodable, 79 Redis key(s) published
push from BSE Providence: 2 frame(s) stored, 0 duplicate(s) (NORAD 62394)
decode: 2 frame(s) read, 2 decoded, 2 written, ... 79 Redis key(s) published
```

A large "not telemetry/undecodable" count is normal for CroCube: image-download passes produce hundreds of 160-byte file chunks per observation that are not telemetry (SatNOGS DB does not decode them either). Only the AX.25 beacons (OBC, PSU, UHF, ...) become points.

5. Command reference

5.1 pvdx-serve

Flag	Effect
<code>--env PATH</code>	dotenv file to load (default <code>.env</code>).
<code>--host / --port</code>	Override <code>API_HOST</code> / <code>API_PORT</code> .
<code>--ingest-poll S / --decode-poll S</code>	Override <code>INGEST_POLL</code> / <code>DECODE_POLL</code> .
<code>--measurement NAME</code>	InfluxDB measurement (default <code>telemetry</code>).
<code>--no-ingest</code>	Do not sweep SatNOGS; frames arrive by push only. <code>NORAD_CAT_ID</code> is then optional.
<code>--no-decode</code>	API and ingest only (no InfluxDB, no Redis).
<code>--no-api</code>	Ingest and decode only.
<code>-v</code>	Debug logging plus the HTTP access log.

5.2 pvdX-push (ground station computer)

Flag / argument	Effect
PATH ...	Frame files (raw bytes, one frame each) or directories of them (one level deep).
--url / --station / --token	Override PUSH_URL / PUSH_STATION / PUSH_TOKEN.
--norad ID	Satellite (default NORAD_CAT_ID).
--time ISO	Receive time for every file; default is the file's modification time.
--frequency HZ	Downlink frequency recorded with every frame.
--meta KEY=VALUE	Extra metadata (repeatable); the file name is always recorded.
--stdin	Read '[ISO-time] hex' lines from stdin instead of files (time defaults to now).
--watch SECONDS	Rescan the given directories forever, sending files not yet sent.

Exit codes: 0 sent, 1 the service rejected the frames or was unreachable after 3 retries, 2 configuration error. 429/5xx/transport errors are retried with backoff; resending a frame is harmless (the service reports it as a duplicate).

5.3 pvdX-ingest

Flag	Effect
--env PATH	dotenv file to load (default .env).
--norad ID	Override NORAD_CAT_ID.
--since ISO	Override INGEST_START for the first sweep of a satellite (no effect once a watermark exists unless it is later than watermark minus overlap).
--state PATH	Override STATE_DB.
--status good bad unknown future failed	Override INGEST_STATUS.
--overlap-hours H	Override INGEST_OVERLAP_HOURS.
--min-interval S	Seconds between list requests (default from the throttle).
--max-pages N	Stop after N pages this run; the sweep resumes next run.
--poll SECONDS	Loop forever, sleeping SECONDS between runs.
--retry-failed	Re-queue frames that exhausted their 5 download attempts.
--stats	Print state-database counters as JSON and exit.
-v	Debug logging (pacing, per-frame lines).

5.4 pvdX-decode

Flag	Effect
--env PATH	dotenv file to load (default .env).
--decoder SPEC	Override DECODER, e.g. satnogs:crocube, satnogs:canvas, pvdX.
--norad ID	Only decode this satellite's frames (default NORAD_CAT_ID; empty means all).

Flag	Effect
--state PATH	Override STATE_DB.
--measurement NAME	InfluxDB measurement (default telemetry). The dashboard expects telemetry.
--limit N	Decode at most N frames this run.
--redo	Re-decode frames already marked decoded (after a decoder change). With NORAD_CAT_ID set, only that satellite's frames.
--dry-run	Decode without writing, marking or publishing anything; prints the first 3 decoded frames of each 500-frame batch as JSON plus the counters.
--poll SECONDS	Loop forever, sleeping SECONDS between runs.
-v	Debug logging (per-frame decode errors).

Exit codes: 0 success, 1 a run finished with errors (frame download failures, InfluxDB unreachable), 2 configuration error, 130 interrupted with Ctrl-C.

6. Viewing output

6.1 The dashboard

Open <http://localhost:3000/d/pvdx-telemetry>. Variables along the top select the NORAD id (discovered from the data), ground stations, and which decoded fields to plot; the voltage and temperature panels are pre-filtered by name pattern. Panels: frames, observations and stations in range; last frame time; frames per hour; battery/bus voltages; temperatures; any field; frames by station; and a table with the latest value of every field with its observation id and station. Values are the decoder's raw units (CroCube reports battery millivolts, for example). The default time range is the last 7 days with auto-refresh every minute; widen it if the last frame is older.

The dashboard is decoder-agnostic: it discovers field names from InfluxDB, so it works unchanged after the PVDX decoder replaces the stand-in. Adjust the regexes of the `voltage_fields` and `temperature_fields` variables in `grafana/dashboards/pvdx-telemetry.json` if PVDX field names use other words.

6.2 The HTTP API

Interactive documentation is at <http://127.0.0.1:8080/docs>. Times are ISO-8601 UTC; `since` and `until` also accept relative forms such as `-6h` or `-2d`. `norad` defaults to `NORAD_CAT_ID`.

Endpoint	Returns
GET /health	ok or degraded; per-satellite counters, watermark and last sweep; worker threads; InfluxDB and Redis reachability.
GET /telemetry	{telemetry: {field: value, alias: value, ...}, stale, frame_time, source}: the flat shape the mission-control backend serves today.
GET /telemetry/latest	Every field with value, frame_time, frame_id, source (satnogs/groundstation) and station_name, plus staleness and the alias map.
GET /telemetry/fields	Known field names, type and when each was last seen.

Endpoint	Returns
GET /telemetry/history?field=&since=&until=&source=&limit=	One field over time, oldest first (the newest limit points).
GET /frames?norad=&since=&until=&source=&station=&decode_status=&limit=&offset=	Frame metadata, newest first.
GET /frames/{id}	One frame: raw_base64, raw_hex, decoded fields, push metadata, observation summary.
GET /observations?norad=&since=&until=&ground_station=&limit=&offset=	SatNOGS observations with per-observation frame counts.
GET /observations/{id}	One observation including the complete SatNOGS record.
POST /ingest/frames	Store frames received by our ground station (section 8.7).

```
curl -s 'http://127.0.0.1:8080/telemetry' | jq .telemetry.battery
curl -s 'http://127.0.0.1:8080/telemetry/history?field=psu_battery&since=-3d' | jq '.points[-1]'
curl -s 'http://127.0.0.1:8080/frames?source=groundstation&limit=5' | jq '.frames[] | {id, frame_time, de
```

6.3 Without Grafana or the API

```
pvdX-ingest --stats # JSON counters and the watermark
pvdX-decode --dry-run --redo --limit 100 # decodes 100 frames; prints first 3 as JSON
sqlite3 data/state.db "select id, source, frame_time, station_name, size, decode_status
from frames order by frame_time desc limit 10"
sqlite3 data/state.db "select field, value, frame_time, source from latest_values"
```

Raw frame bytes never leave SQLite: frames.raw holds them with source (satnogs or groundstation), sha256, size, the frame time (from the SatNOGS file name or the pushed receive time), push metadata, download/decode bookkeeping, the decoder name and the decoded fields as JSON. latest_values keeps the newest value of every field per satellite and only moves forward in frame time. observations holds station id, name, position, start/end, observer, transmitter, frequency, TLE lines and the complete API record as JSON.

7. Data model in InfluxDB and Redis

Item	Value
Bucket / measurement	telemetry / telemetry (both hardcoded as constant variables in the dashboard JSON)
Tags	norad_cat_id, sat_id, decoder, source (satnogs/groundstation), observation_id (empty for pushed frames), ground_station, station_name
Fields	every decoded value (numbers as floats, strings as strings) plus frame_id
Time	frame time from the SatNOGS file name (fallback: observation start), nanosecond precision with the frame id as sub-second offset so same-second frames stay distinct and re-writes stay idempotent

A frame is marked decoded only after its point is written. An InfluxDB outage (5xx, connection refused) or a credential/bucket problem (401/403/404) leaves the batch undecoded to be retried on the next run; a point InfluxDB refuses because of its data (400/422, for example a field type conflict) is marked error with the reason and does not block the rest of the batch.

Redis (when REDIS_URL is set) holds the latest values for the web app backend, all with the same TTL:

Key	Value
telemetry:<field>	Newest value of each decoded field as a string (8160, SAFE).
telemetry:<alias>	The same values under the names from TELEMETRY_ALIASES (battery, temperature, ...).
telemetry:_meta	JSON: satellite, newest frame time/id/source/station, published_at, aliases, TTL.
telemetry:_all	JSON: every field with value, frame_time, frame_id, source and station_name.

The keys are rewritten after every decode pass (and once at startup from latest_values), so a Redis outage never loses anything: the next pass republishes the full set.

8. Routine operations

8.1 Change the satellite

```
# .env
NORAD_CAT_ID=68635
DECODER=satnogs:canvas
# then simply run pvdx-ingest / pvdx-decode; state and watermarks are per NORAD id
```

Any satellite with a struct in the satnogs-decoders package works; the DB name of the decoder (as shown on <https://db.satnogs.org>) is the struct name, case-insensitive. The survey table in README.md lists the most active candidates measured on 2026-09-26.

8.2 Re-decode after a decoder change

```
pvdx-decode --redo          # re-decode NORAD_CAT_ID's frames; same decoder overwrites
# Switching to a different DECODER name? Delete the old points first (8.5):
# "decoder" is a tag, so the dashboard would otherwise show both series.
```

8.3 Recover frames that failed to download

```
pvdx-ingest --stats          # frames_failed > 0 ?
pvdx-ingest --retry-failed   # resets their attempt budget, then runs a normal sweep
```

8.4 Rotate the SatNOGS token

Edit SATNOGS_API_TOKEN in .env and restart pvdx-ingest. A rejected token shows up once as an ERROR line and the run continues anonymously. If the variable is also exported in your shell, unset it or open a new terminal: an exported environment variable always wins over .env.

8.5 Reset everything

```
# stop any running pvdx-ingest / pvdx-decode first (Ctrl-C)
docker compose down -v          # deletes InfluxDB + Grafana data (and their credentials)
rm -f data/state.db data/state.db-wal data/state.db-shm
docker compose up -d
pvdx-ingest                     # exit code 1 only means some frame downloads failed
pvdx-decode
```

To clear only the InfluxDB points but keep the SQLite frames, delete the measurement and re-decode. Never source .env in your working shell: exported variables take precedence over the file for both the Python commands and docker compose, so later .env edits would be silently ignored in that shell.

```
( set -a; source .env; set +a      # subshell: keeps .env values out of your working shell
body='{ "start": "1970-01-01T00:00:00Z", "stop": "2100-01-01T00:00:00Z", '
body="$body" '"predicate": "_measurement=\"telemetry\""'
curl -X POST -H "Authorization: Token $INFLUX_TOKEN" \
-H "Content-Type: application/json" -d "$body" \
"localhost:8086/api/v2/delete?org=$INFLUX_ORG&bucket=$INFLUX_BUCKET" )
pvdx-decode --redo
```

8.6 Connect the mission-control web app

The pvdx-mission-control backend reads telemetry:battery, elevation, temperature, signal_rssi and uptime_seconds from Redis and reports stale when a key is missing. Point its Redis client at the Redis this service publishes to, delete its fake telemetry loop, and set TELEMETRY_ALIASES here so those names map to real decoded fields (.env.example has the CroCube mapping; values are raw decoder units, e.g. battery in mV). Its GET /telemetry then returns real data with no other change. Alternatively the frontend can call this service's GET /telemetry directly (same shape); add its origin to API_CORS_ORIGINS.

8.7 Push frames from the ground station

Install this package on the ground station computer, set PUSH_URL (the service), PUSH_STATION (this station's name), PUSH_TOKEN (equal to the service's INGEST_TOKEN) and NORAD_CAT_ID in its .env, then:

```
pvdx-push frame1.bin frame2.bin          # one raw frame per file; receive time = file mtime
pvdx-push --time 2026-09-27T14:03:05Z --frequency 436500000 --meta pass=12 frame.bin
pvdx-push --watch 5 /var/lib/gnuradio/frames/ # keep sending new files from a directory
printf '2026-09-27T14:03:05Z 86a2...\n' | pvdx-push --stdin # "[ISO-time] hex" lines
```

Any HTTP client can do the same: POST /ingest/frames with header X-Ingest-Token and the JSON body {norad_cat_id, station, frames: [{raw (base64), received_at, frequency?, rssi?, meta?}]]; at most 1000 frames per request and 64 KiB per frame. Pushed frames are keyed by (satellite, station, receive second, sha256), so resending is harmless. They are decoded within about a second and show up in the API, Redis and InfluxDB with source = groundstation and the station name.

8.8 Upgrade from 0.1

Install the new version (uv pip install -e ".[dev]"), add the new keys from .env.example to .env (all have working defaults) and start pvdx-serve. The state database is migrated in place on first open (a backup copy of data/state.db first is cheap); already-decoded frames are queued again so their decoded fields get stored, which rewrites identical InfluxDB points. Downgrading afterwards is not supported; the old code refuses a newer schema.

8.9 Back up

Copy data/state.db (with sqlite3 data/state.db ".backup backup.db" while the pipeline runs) and, if wanted, the Docker volumes bse_code_influxdb-data and bse_code_grafana-data. InfluxDB can always be rebuilt from state.db with pvdx-decode --redo.

9. Swapping in PVDX

- Set NORAD_CAT_ID to PVDX's catalog number (or the temporary id SatNOGS assigns after launch) and DECODER=pvdx in .env.

- Implement `PvdxDecoder.decode` in `pvdx_ground/decode/pvdx.py`: parse the USLP transfer frame, extract the Space Packet(s), map each APID to fields, and return a flat `dict[str, float | int | str]`. The stub currently raises `DecodeError` and lists the steps as TODO comments. If PVDX instead gets a Kaitai struct merged into `satnogs-decoders`, `DECODER=satnogs:pvdx` needs no code.
- Record a real PVDX frame under `tests/fixtures/frames/` and add a decode test next to the CroCube ones in `tests/test_decode_storage.py`.
- Set `TELEMETRY_ALIASES` so the web app's names map to PVDX field names.
- Grafana needs no change; tune the `voltage_fields` / `temperature_fields` regexes if desired. The push path is decoder-agnostic: the ground station sends whatever bytes it demodulated.

10. Troubleshooting

Symptom	Cause and fix
ERROR ... rejected SATNOGS_API_TOKEN (401 Invalid token)	The token is a SatNOGS DB token or is wrong. Ingest continues anonymously at 60 req/h. Create a Network API key (section 2.3).
WARNING ... HTTP 429; retry n/6 in Ns	Throttled. The client honours Retry-After and continues; nothing to do. If it repeats, another process on the same IP is also querying SatNOGS, or <code>INGEST_MIN_INTERVAL</code> was lowered.
configuration error: NORAD_CAT_ID is not set	<code>.env</code> is missing, not in the working directory, or the value is empty. Run from the repository root or pass <code>--env</code> / <code>--norad</code> . Exit code 2.
configuration error: no decoder configured / has no struct named ...	<code>DECODER</code> is empty or misspelt. Set <code>DECODER=satnogs:<struct></code> (name as on <code>db.satnogs.org</code> , case-insensitive) or <code>pvdx</code> , or pass <code>--decoder</code> . Exit code 2.
HTTP 400 ... (from the API)	<code>--status</code> and <code>--since</code> are validated locally (exit 2 with a 'configuration error' line), so a real HTTP 400 points at <code>SATNOGS_NETWORK_URL</code> or an API change. A rejected saved cursor is handled automatically (next row).
saved cursor for sweep N rejected ... aborting	SatNOGS no longer accepts an old cursor. The sweep is dropped (that run exits 1; under <code>--poll</code> the next poll retries) and the next run starts a fresh sweep from watermark minus overlap (from <code>INGEST_START</code> if no sweep has completed yet). Just run again.
sqlite3.OperationalError: database is locked	Another connection held the write lock for over 30 s. <code>pvdx-ingest</code> and <code>pvdx-decode</code> only use short per-observation/per-frame transactions, so look for a manual <code>sqlite3</code> session left inside a transaction, or a <code>state.db</code> on a network share. Normal ingest + decode concurrency is fine.
InfluxDB not reachable ... / InfluxDB rejected the credentials (401) / bucket 'telemetry' does not exist	Startup check of <code>pvdx-decode</code> . Stack down or still starting: <code>docker compose up -d</code> and wait for (healthy). 401: <code>INFLUX_TOKEN/INFLUX_ORG</code> in <code>.env</code> differ from what the InfluxDB volume was initialised with. Missing bucket: <code>INFLUX_BUCKET/INFLUX_ORG</code> differ. Fix <code>.env</code> or reset the volumes (8.5).
INFLUX_TOKEN is not set	Add <code>INFLUX_TOKEN</code> to <code>.env</code> (must match what the InfluxDB container was initialised with; otherwise reset the volumes).
Dashboard shows No data everywhere	No points yet: run <code>pvdx-decode</code> . If the NORAD ID dropdown is empty, no point is newer than 30 days (the variable queries use a fixed 30-day look-back, independent of the time picker): ingest and decode newer data, or edit <code>start: -30d</code> in the variable queries. If the dropdown is populated but panels are empty, widen the time range (default: last 7 days).

Symptom	Cause and fix
Dashboard variables show a warning triangle	Grafana cannot query InfluxDB. Check INFLUX_TOKEN in .env matches what the InfluxDB volume was initialised with, then docker compose up -d grafana to recreate the container (docker compose restart does not re-read .env). The datasource is provisioned read-only, so it cannot be fixed from the Grafana UI.
docker: unknown command: docker compose	Docker Desktop is installed but was never started, so the Compose plugin is not linked. Start Docker Desktop once, then retry.
Grafana asks to update the password on every login	Expected with the default admin password; click Skip or set a password (it is stored in the grafana-data volume).
Many frames marked error with UnicodeDecodeError / EOFError	Non-telemetry frames (image chunks, digipeater packets, truncated captures). Expected; only beacons decode.
pvdX-push: HTTP 401 missing or invalid X-Ingest-Token	PUSH_TOKEN on the station differs from INGEST_TOKEN on the service (or the service has none and the client sends one is fine; the reverse is not). Align the two.
WARNING INGEST_TOKEN is not set: POST /ingest/frames accepts frames from anyone	Fine on a laptop. Set INGEST_TOKEN before exposing the service beyond localhost.
Redis publisher: Redis at ... not reachable / could not publish latest telemetry	Redis is down or REDIS_URL is wrong. Decoding continues; keys are republished on the next pass once Redis answers. docker compose up -d redis.
GET /health returns degraded with workers.decode = stopped	The decode (or ingest) thread hit an unrecoverable error; see the log line 'worker stopped after an unrecoverable error' above it. Fix the cause and restart pvdX-serve.
[Errno 48] address already in use (uvicorn)	Another pvdX-serve (or the pvdX container) already listens on API_PORT. Stop it or pass --port.
docker: RWLayer of container ... is unexpectedly nil	Docker Desktop lost track of an old container. docker rm -f pvdX-influxdb pvdX-grafana, then docker compose up -d; the data volumes are untouched.
GET /telemetry shows stale: true although frames arrive	The newest decoded frame is older than TELEMETRY_STALE_AFTER, or the pushed received_at times are wrong (they must be UTC; naive times are taken as UTC). Check /telemetry/latest newest_frame_time.

11. Maintenance and tests

```

pytest # 80 tests on recorded pages + real frames; no network
uv pip install -U satnogs-decoders # newer Kaitai structs (after an upstream decoder fix)

```

Test fixtures live in tests/fixtures/: two consecutive observation pages with their real Link headers, eleven frame files from those pages, and four real CroCube frames with their expected decode. The API is tested with FastAPI's test client, Redis with an in-memory stand-in, and pvdX-push against a mock service; the decode loop's wake-up on push and its behaviour during a Redis outage are covered too. To refresh them, record new pages with curl, update manifest.json, the .link files and the NORAD/SINCE constants in tests/conftest.py, and revisit the assertions that encode the recorded set's shape (two pages of 25 plus an empty last page, 11 frames of which 2 on page 1, one page-1 observation without demodata) in tests/test_worker.py and tests/test_client.py.

12. Verified SatNOGS API facts

These were checked against the live API and the satnogs-network source on 2026-09-26 and differ from older documentation:

- The satellite filter is `norad_cat_id`; `satellite__norad_cat_id` is silently ignored and returns every satellite.
- `status` is a string (failed, bad, unknown, future, good); integers return HTTP 400.
- The observations endpoint returns a bare JSON list of 25 items; the next page is only advertised in the `Link: <url>; rel="next"` header. Ordering is newest first.
- `start` means `start >= value`, `end` means `end <= value`; `start__lt`, `end__gt` and comma-separated `observation_id` also exist.
- Throttle: 60 list requests per hour anonymous, 240 with a Network token; 429 responses carry `Retry-After`. Frame files on object storage are not throttled.
- The OpenAPI schema URL in the original brief (`network.satnogs.org/api/schema/`) is a 404; SatNOGS DB publishes one at `db.satnogs.org/api/schema/?format=json`.

13. Known gaps

- Backfill speed is bounded by the SatNOGS throttle (about 1 500 observations per hour anonymously).
- Observations vetted good later than the overlap window are missed. Moving `INGEST_START` back does not help once a watermark exists (every sweep starts at `max(INGEST_START, watermark minus overlap)`); run one sweep with a larger overlap, e.g. `pvdX-ingest --overlap-hours 720`, or delete the satellite's row in the `watermarks` table.
- Kaitai structs are permissive: garbage bytes can decode into nonsense values. No range checks yet.
- All numbers are written as floats; enum-like fields appear as strings and are not plottable.
- One satellite per `pvdX-serve` process, and the Redis keys are not prefixed by satellite; pushed frames for another satellite are stored but not decoded until that id is configured.
- The PVDX decoder is a stub until the USLP/SPP format is frozen.
- No authentication on the API (left to the web app layer); only the push endpoint has a shared secret. Put the service behind the backend or a reverse proxy before exposing it.
- Not built (phase 2): scheduling observations on the SatNOGS Network, CFDP reassembly of downlinked files/photos, and the commanding/uplink path.